Traitement et Analyse de données

Introduction

Romain Negrel
romain.negrel@esiee.fr

ESIEE Paris

septembre 2026

Université Gustave Eiffel

ESIEE PARIS

Informations

Note finale

- 1 ou 2 QCM
- 1 Projet noté

Prérequis

- Algèbre linéaire
- Programmation Python
- Notebook jupyter

Approche

Ce cours privilégie les méthodes **statistiques classiques** (antérieures au *deep learning*), pour bien comprendre les concepts fondamentaux avant d'aborder des modèles plus complexes.

Sommaire

1. Introduction
2. Approche générale
3. Mise en œuvre
4. Formulation & Optimisation
5. Généralisation et évaluation

Introduction

1. Introduction
2. Approche générale
3. Mise en œuvre
4. Formulation & Optimisation
5. Généralisation et évaluation

L'informatique

Objectif majeur

Effectuer des **tâches** et **résoudre des problèmes** de manière automatique

Comment ?

1. Un ordinateur est capable d'exécuter des **milliards d'instructions par seconde**
2. **Un humain écrit un algorithme** pour résoudre une problème

Exemples

- Déterminer la plus petite valeur entre deux valeurs
- Calcul du PGCD de deux nombres (Euclide 300 avant J.-C.)
- Tri d'un tableau
- Chemin dans un graphe

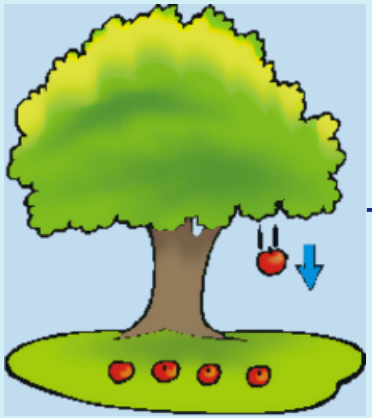
La prédiction

Définition

Annonce d'événements futurs par la connaissance qu'on a de leurs causes

Modèle mathématique

Un humain crée le modèle mathématique qui décrit le phénomène
Exemple :



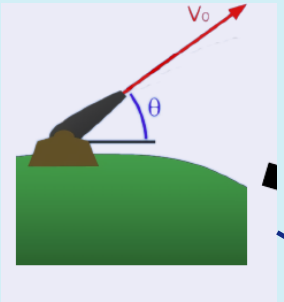


Équations
Mécanique Newtonienne

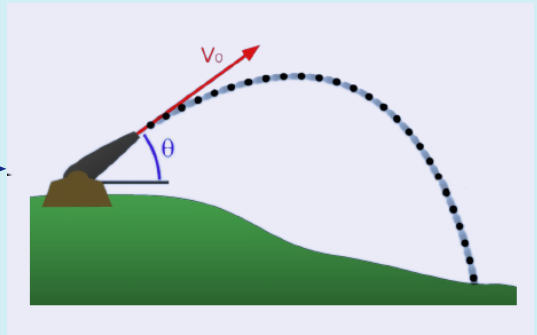
La prédiction

Exemple de prédiction

La Balistique





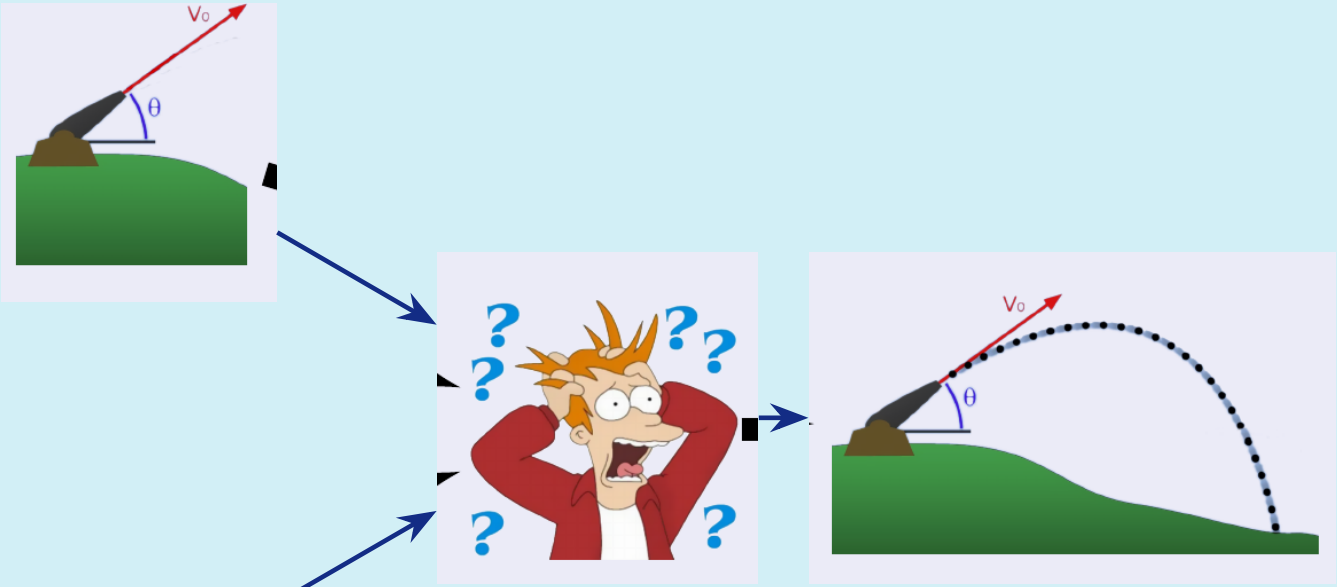


Équations
Mécanique Newtonienne

La prédiction

Exemple de prédiction

La Balistique



Équations
Mécanique Newtonienne

Première problématique

Comment faire quand **les calculs** sont **trop complexe** pour un Humain ?

La prédiction

Solution

Automatisé les calculs !
Exemple, la prédiction météorologique



La prédiction

Solution

Automatisé les calculs !
Exemple, la prédiction météorologique



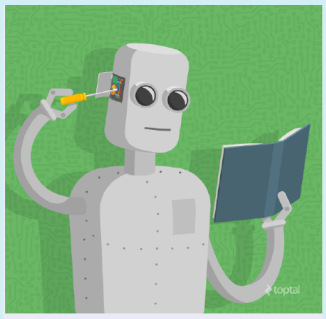
Deuxième problématique

Comment faire quand le **modèle mathématique** sont **trop complexe** pour être trouvé par un Humain ?

La prédiction

Solution

Machine Learning
ou
Apprentissage Automatique



Définition : Ensemble des méthodes permettant à une machine à résoudre un **problème automatiquement** à partir d'un ensemble d'**exemples de données**

Exemples vs Prédictions

Exemples : les données connues, utilisées par la machine pour **apprendre** (son « entraînement »).
Prédictions : la réponse que la machine produit ensuite sur de **nouvelles** données — pas toujours une prévision au sens strict : une catégorie, un groupe, une valeur manquante...

Exemples d'application

liés à notre quotidien :

- trier des mails par thème, filtrer les spams *supervisé*
- regrouper les acheteurs par types *non supervisé*
- prédire le prix de l'immobilier, le nombre de ventes d'un nouveau produit *supervisé*
- prédire la hausse ou la baisse d'un cours boursier *supervisé*

plus techniques :

- détecter des transactions bancaires frauduleuses inhabituelles *non supervisé*
- faire un diagnostic automatique à partir d'une analyse médicale *supervisé*
- extraire et reconnaitre du texte ou un visage dans une image *supervisé*
- classer les photos d'un album par personne, avec seulement quelques photos annotées *semi-supervisé*

Apprentissage supervisé (*supervised learning*)

Principe

Les **prédictions attendues sont prédéterminées** : la base d'apprentissage contient des couples (donnée, prédiction attendue) — on parle de **vérité terrain** (*ground truth*) ou de données **labellisées**.

Deux grandes applications

- **La régression** : le paramètre de sortie est de type quantitatif ;
- **La discrimination** (*Classification*) : le paramètre de sortie est de type catégoriel.

Exemples

- **Prédiction de l'age** : entrée = photo d'identité, sortie = âge en années (régression) ;
- **Filtre de SPAMS** : entrée = email, sortie = SPAM ou NON-SPAM (discrimination).

Apprentissage non supervisé (*unsupervised learning*)

Principe

Les **prédictions attendues sont inconnues** : la base d'apprentissage ne contient **que des données**, sans vérité terrain associée.

Deux grandes applications

- **La catégorisation** (*Clustering*) : les données sont regroupées en groupes homogènes;
- **La détection d'anomalie** : détection des données qui sont anormales par rapport à l'ensemble d'apprentissage.

Exemples

- **Gestion de la relation client** : regrouper les clients en groupes homogènes pour un service personnalisé;
- **Alarme de vidéosurveillance** : détection d'événement rare (accident, mouvement de foule, agression, *etc*) à partir du flux vidéo.

Apprentissage semi-supervisé (*semi-supervised learning*)

Principe

Une **petite partie** des données est labellisée, une **grande partie** ne l'est pas — on combine les deux pour apprendre. Utile car labelliser coûte cher (temps humain), alors que les données brutes sont abondantes.

Exemple

Classer les photos d'un album par personne : seules quelques photos sont annotées manuellement, le reste est déduit automatiquement.

Apprentissage auto-supervisé (*self-supervised learning*)

Principe

Aucune étiquette humaine : le signal de supervision est fabriqué **automatiquement** à partir de la structure des données elles-mêmes — une partie de la donnée sert à prédire une autre partie. *À ne pas confondre avec le semi-supervisé*, qui utilise lui un (petit) nombre d'étiquettes humaines. Usage typique : **pré-entraînement** massif sur d'énormes quantités de données non annotées $\rightarrow$ puis **affinage** (*fine-tuning*) supervisé sur une petite base labellisée pour la tâche finale.

Exemple

Deviner le mot masqué dans une phrase à partir du contexte, ou prédire le mot suivant : c'est le principe derrière les grands modèles de langage (GPT, BERT, ...). Le « label » (le mot caché) est connu automatiquement — aucune annotation humaine nécessaire.

D'autres types d'apprentissage

Panorama non exhaustif, hors programme de cette UE :

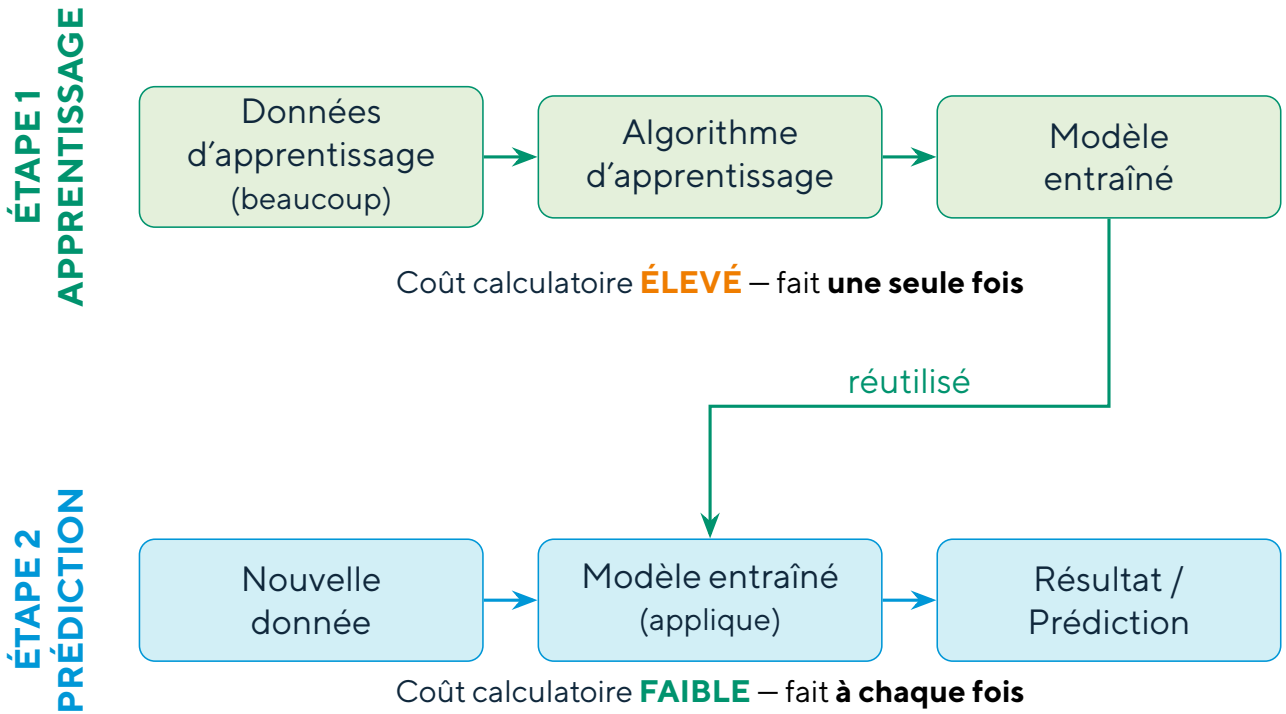
- **Apprentissage par renforcement** (*reinforcement learning*) : la machine apprend par essai-erreur, guidée par une récompense — ex. jeux, robotique.
- **Apprentissage par transfert** (*transfer learning*) : réutiliser un modèle déjà entraîné sur une tâche proche, plutôt que repartir de zéro.
- **Apprentissage few-shot / zero-shot** : généraliser à une nouvelle catégorie à partir de très peu (voire aucun) exemple étiqueté.

Dans cette UE

- Cette UE se concentre sur l'**apprentissage supervisé** : régression (cours 3), régression logistique (cours 4), discrimination bayésienne (cours 5).
- Un exemple d'apprentissage **non supervisé** : le K-Means (cours 2, avec le sac de mots).
- Semi-supervisé, auto-supervisé, par renforcement : mentionnés pour le panorama, **hors programme** de cette UE.

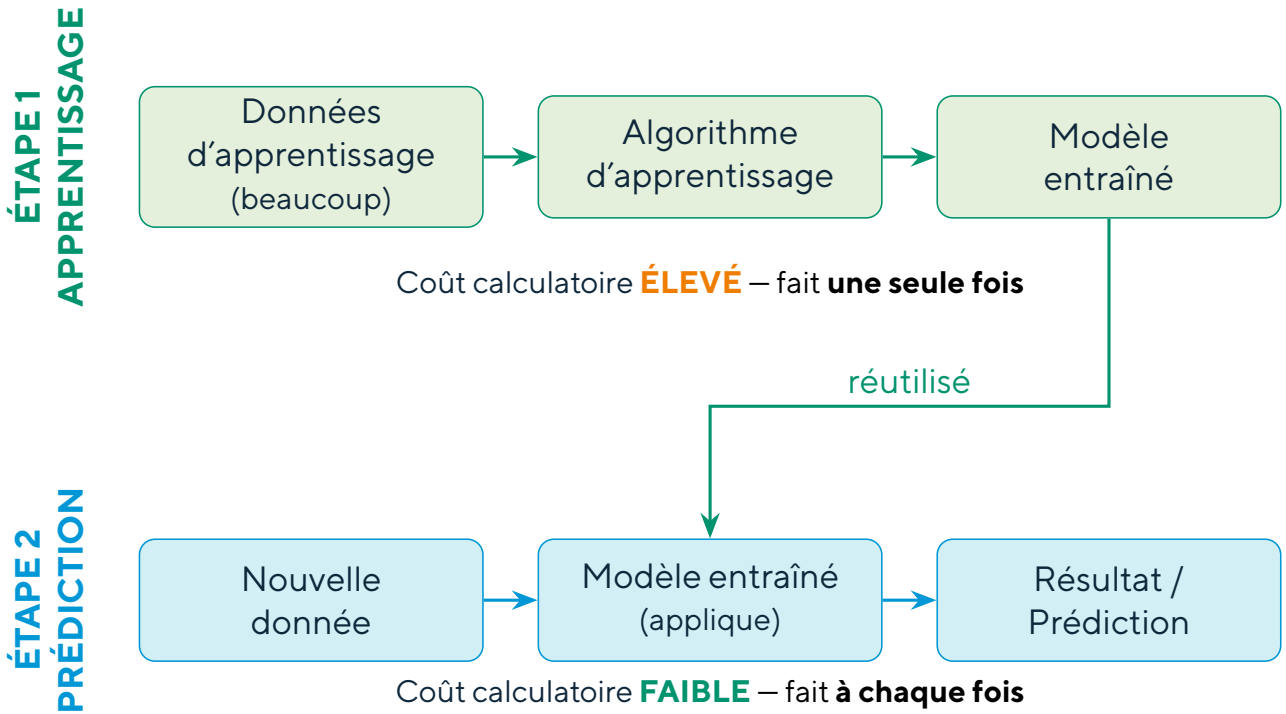
Idée principale

Quelle que soit la catégorie (supervisé, non supervisé...), apprendre de la **connaissance** à partir de **données** se fait toujours en **deux étapes** :



Idée principale

Quelle que soit la catégorie (supervisé, non supervisé...), apprendre de la **connaissance** à partir de **données** se fait toujours en **deux étapes** :



Deux questions restent ouvertes

- Comment sait-on que le modèle entraîné **fonctionne vraiment bien** ?
- Comment **choisir** la bonne méthode et les bons hyperparamètres ?

Approche générale

1 Introduction

2 Approche générale

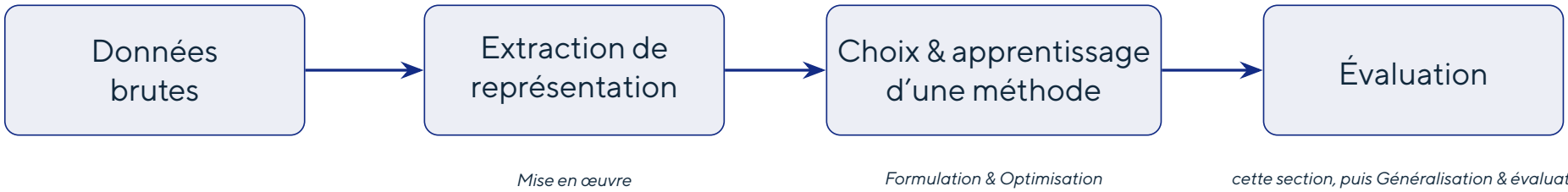
3 Mise en œuvre

4 Formulation & Optimisation

5 Généralisation et évaluation

La chaîne de traitement

Toute méthode de machine learning suit le même schéma général :



- **Données brutes** : texte, image, son...souvent inexploitables telles quelles.
- **Extraction de représentation** : transformer en vecteurs numériques.
- **Choix & apprentissage d'une méthode** : plusieurs méthodes possibles pour un même problème, chacune avec ses propres réglages.
- **Évaluation** : mesurer si ça marche vraiment, et généralise à de nouvelles données.

Qu'est-ce qui est appris ?

Revenons sur l'**ÉTAPE 1 — APPRENTISSAGE** (vue en Introduction) : tout ce qui doit être fixé avant de passer à l'exploitation ne l'est pas de la même façon.

Deux endroits où quelque chose se fixe

- Dans **l'extraction** : ex. un dictionnaire, une normalisation.
- Dans **la méthode** elle-même : ex. les coefficients d'une droite de régression.

Deux façons de le fixer

Parfois **automatiquement**, en minimisant un critère (ex. descente de gradient) ; parfois par **essais successifs** (ex. validation croisée) — mais dans les deux cas, c'est **appris** avant l'exploitation.

Est-ce que tout se fixe de la même façon ?

Non — d'où la distinction qui suit.

Paramètres vs hyperparamètres

Les paramètres

Ce que le modèle **apprend automatiquement** à partir des données d'entraînement (ex. les coefficients d'une droite de régression).

Les hyperparamètres

Des choix fixés **avant** l'apprentissage, qui ne sont **pas** appris automatiquement :

- le degré d'un polynôme, le nombre de voisins k...
- la force d'une éventuelle régularisation
- **le choix de la méthode elle-même** ! Régression vs k-NN vs réseau de neurones...chaque méthode a en plus ses propres hyperparamètres.

Question ouverte

Comment choisir ces hyperparamètres, puisqu'ils ne sont pas appris automatiquement ?

Pourquoi ne pas tout apprendre automatiquement ?

Une idée tentante

Puisque les paramètres sont appris en minimisant un critère sur les données d'apprentissage, pourquoi ne pas y mettre aussi les hyperparamètres, et tout optimiser d'un coup ?

Le piège

Un critère qui s'évalue sur les **mêmes** données qu'il utilise pour apprendre a toujours intérêt à devenir **plus complexe** : augmenter le degré D d'un polynôme ne peut que **faire baisser** l'erreur sur ces données — jusqu'à, dans le pire des cas, passer exactement par tous les points.

Le problème

Coller parfaitement aux données déjà vues n'est **pas** le but : ce qu'on veut, c'est bien fonctionner sur de **nouvelles** données. Il faut donc évaluer les hyperparamètres **ailleurs** que sur les données d'apprentissage.

Train / Test — la première idée

Répondre à la première question

« Comment sait-on que le modèle fonctionne vraiment bien ? »
On réserve une partie des données, jamais utilisée pendant l'apprentissage, pour vérifier.

Train
apprend le modèle

Test
jamais vu, vérifie

« Fonctionne bien »

Ça veut dire : fonctionne sur **test**, pas seulement sur **train**. Si l'écart entre les deux est grand, il y a un problème (sur-apprentissage).

La limite de Train/Test seul

Et pour répondre à la deuxième question ?

« Comment choisir la bonne méthode et les bons hyperparamètres ? »
Idée naïve : essayer plusieurs choix, garder celui qui a la meilleure performance sur **test**.

La limite de Train/Test seul

Et pour répondre à la deuxième question ?

« Comment choisir la bonne méthode et les bons hyperparamètres ? »
Idée naïve : essayer plusieurs choix, garder celui qui a la meilleure performance sur **test**.

Problématique

En choisissant **d'après le score sur test**, on a implicitement adapté nos choix **pour** le test. Le score obtenu n'est alors plus une estimation honnête de la performance réelle — on a « triché » sans même avoir utilisé les données de test pour apprendre.

Solution

Il faut un **troisième ensemble**, dédié aux choix (hyperparamètres), différent de celui réservé à l'évaluation finale.

Ensemble d'entraînement, de validation et de test

Découpage en trois

- **Entraînement** (train) : sert à apprendre — l'extraction de caractéristiques (ex. un dictionnaire de mots, une normalisation) **et** les paramètres du modèle.
- **Validation** : on **réapplique** (sans réapprendre) l'extraction de caractéristiques et le modèle appris sur train, on mesure la performance $\rightarrow$ sert à choisir les **hyperparamètres**.
- **Test** : mis de côté dès le départ, regardé **une seule fois à la fin** avec l'extraction de caractéristiques et le modèle finaux (jamais réappris) $\rightarrow$ performance reportée.

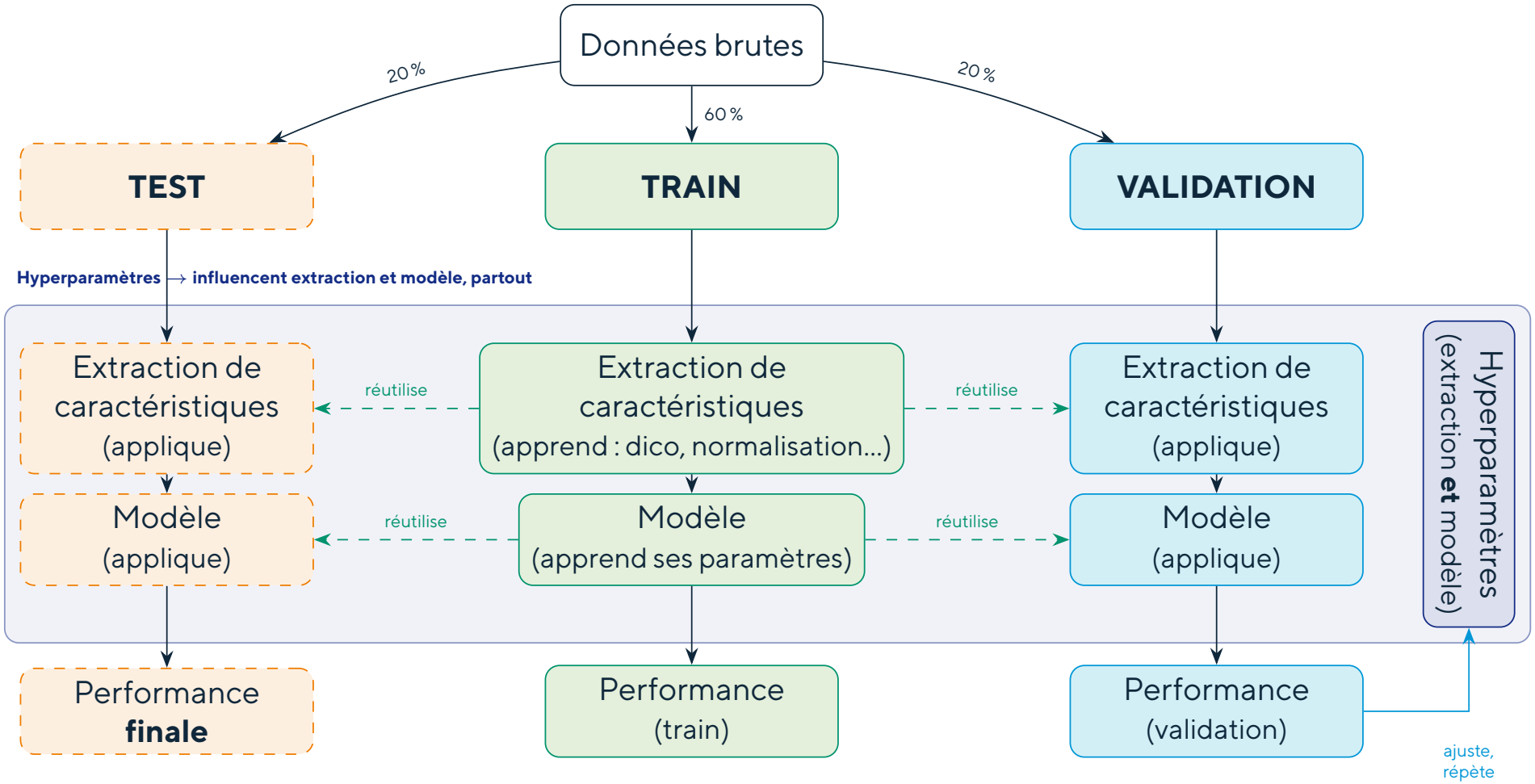
Point clé

Ce n'est pas seulement le modèle qui est appris sur train : l'extraction de caractéristiques aussi. Ce qui est appris sur train est **réutilisé tel quel** sur validation et test, jamais réappris.

L'extraction de caractéristiques a aussi des hyperparamètres

Ex. taille d'un vocabulaire de mots, nombre de bins d'un histogramme, nombre de composantes (PCA)...Ils se choisissent **eux aussi** sur validation, exactement comme les autres.

Le découpage train / validation / test



Test : évaluation finale, une seule fois.
Train et validation très différents ⇒ signal d'alerte (ex. sur-apprentissage), **avant même** de regarder le test.

Train : apprend paramètres **et** teste des hyperparamètres.

Validation : choisit les hyperparamètres (répété).

Validation croisée (k-fold)

Le problème d'un seul découpage

Avec peu de données, la performance mesurée sur **une seule** validation dépend beaucoup du tirage aléatoire — mauvaise estimation.

itér. 1	val	train	train	train	train
itér. 2	train	val	train	train	train
itér. 3	train	train	val	train	train
itér. 4	train	train	train	val	train
itér. 5	train	train	train	train	val

- On découpe l'ensemble d'apprentissage en k parts égales ($k = 5$ ci-dessus).
- On répète k fois : $k - 1$ parts pour l'entraînement, la dernière pour la validation (à tour de rôle) — puis on **moyenne** les k performances.
- Utilise toute la donnée pour la validation, sans jamais toucher au test. Surtout utile avec **peu de données** (coût : k fois plus d'entraînements).

Et comment mesure-t-on la performance ?

Il faut un critère chiffré

Train, validation, test ne servent à rien si on ne sait pas **quoi mesurer** : il faut choisir une **métrique**, adaptée au problème.

Quelques exemples

- Régression (sortie quantitative) : une erreur numérique, ex. l'erreur quadratique moyenne.
- Classification (sortie catégorielle) : un taux de bonnes réponses, ex. le taux d'erreur.

Métrique $\neq$ loss d'entraînement

La métrique la plus parlante pour **comparer** n'est pas toujours utilisable pour **optimiser** (ex. non dérivable) : on entraîne alors sur une **loss** différente, et on choisit/valide sur la métrique qui nous intéresse vraiment — les deux peuvent diverger.

L'erreur quadratique moyenne est détaillée juste après; les critères pour la classification (matrice de confusion, etc.) seront vus en fin de cours.

Risque empirique et risque réel

Risque empirique

Ce qu'on peut **calculer**, sur les données d'apprentissage (P exemples) :

$$R_{\text{emp}}(f) = \frac{1}{P} \sum_{i=1}^{P} L(f(x_i), y_i)$$

Risque réel (théorique)

Ce qu'on voudrait minimiser, sur **toute** la distribution des données (inconnue) :

$$R(f) = \mathbb{E}_{(x,y)} \left[L(f(x), y) \right]$$

Généraliser

C'est faire en sorte que $R_{\text{emp}}(f)$ (calculable) soit un **bon indicateur** de $R(f)$ (ce qu'on veut vraiment). L'écart entre les deux est ce que toute cette section va chercher à comprendre et à contrôler.

- L : taux d'erreur, erreur quadratique moyenne, probabilité d'erreur, *etc.*
- On l'estime en évaluant sur des exemples qui n'ont **pas servi** à l'apprentissage.

Sous-apprentissage et Sur-apprentissage

Il y a sous-apprentissage quand

la méthodes d'apprentissage n'a pas la capacité d'apprendre correctement par rapport à la complexité des données.

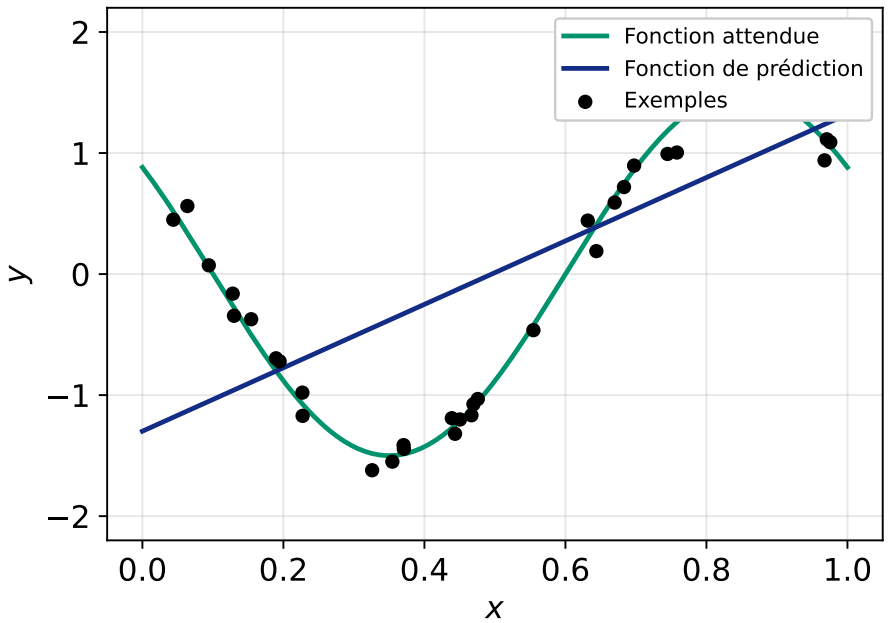


Figure : régression linéaire (degré 1) sur des données non linéaires

- Le modèle est trop simple pour représenter la courbe.
- Erreur élevée même sur les données d'**entraînement**.

Sous-apprentissage et Sur-apprentissage

Il y a Sur-apprentissage quand

la méthodes d'apprentissage apprend par cœurs les données d'entrainement et ce trompe sur les données de teste

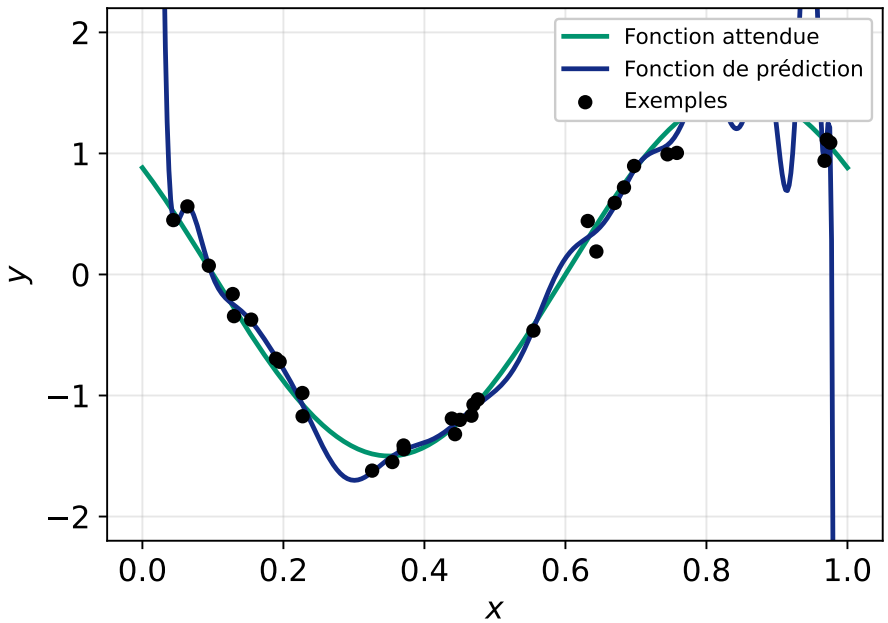


Figure : régression polynomiale (degré élevé), peu d'exemples

- Le modèle passe par (presque) tous les points d'entraînement.
- Il oscille violemment ailleurs : il a mémorisé le bruit, pas la tendance.

Sous-apprentissage et Sur-apprentissage

Comment choisir ?

Pour choisir la bonne méthode d'apprentissage, il faut étudié ces performances sur des exemples qui n'ont pas servi a l'apprentissage !

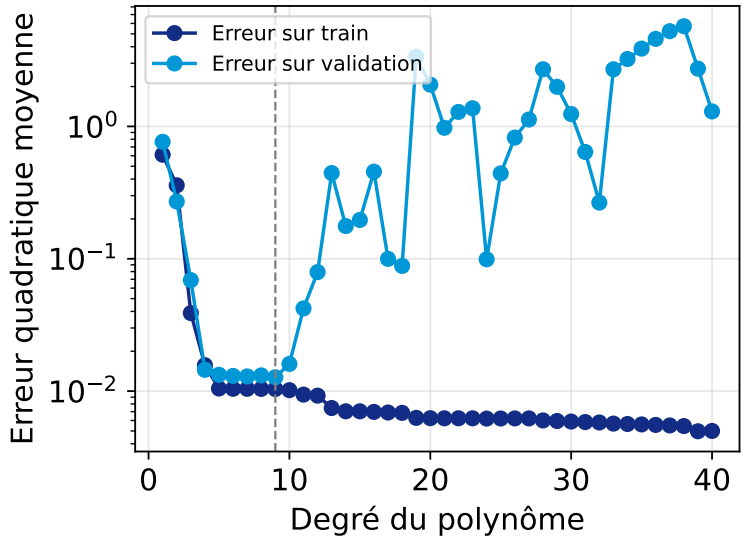


Figure : erreur vs degré

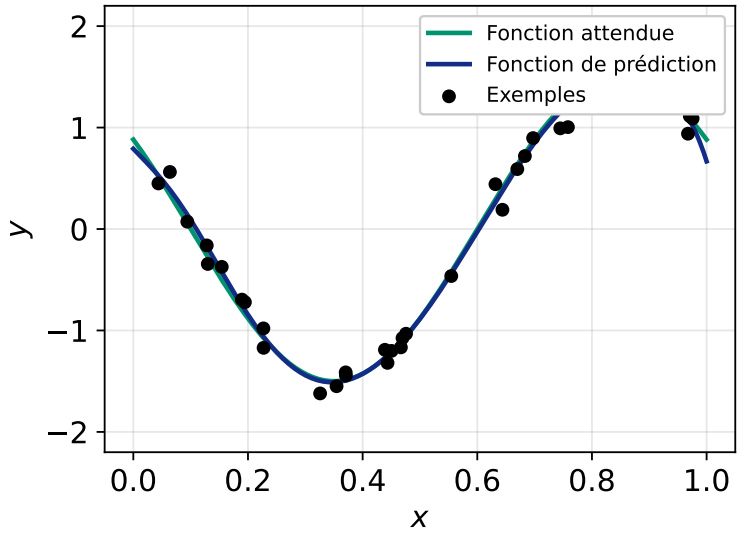


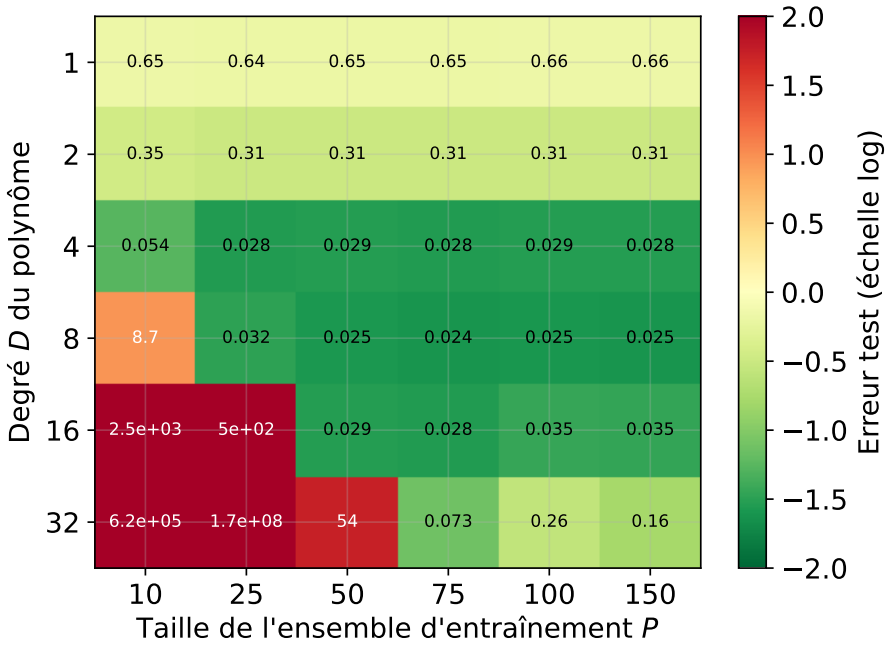
Figure : degré optimal (min. erreur **validation**)

- L'erreur d'entraînement baisse toujours avec le degré (bleu) : elle ne dit rien sur le sur-apprentissage.
- L'erreur de **validation** (cyan) forme un creux : à gauche on sous-apprend, à droite on sur-apprend. Le bon degré est choisi **sur cette courbe**, pas sur l'erreur d'entraînement, ni sur le test (réservé à la toute fin).

Effet du volume de données

Erreur de test, selon le degré D et la taille P de l'entraînement

Chaque case : un modèle de degré D (ligne) entraîné sur P exemples (colonne).



- À P fixé, un degré trop élevé explose (rouge) : sur-apprentissage.
- Mais à degré **fixé, même très élevé** (ex. $D = 32$), l'erreur redevient bonne (vert) quand P augmente : **le volume de données compense la complexité**.
- Un modèle simple ($D = 1$ ou 2) ne s'améliore **jamais** beaucoup, quel que soit P : plus de données ne corrige pas un sous-apprentissage.

Malédiction de la dimension

Plus de dimensions, plus de données nécessaires

En haute dimension, les données deviennent **éparses** : le volume à couvrir croît **exponentiellement** avec le nombre de dimensions N.

Exemple

Pour couvrir un espace en dimension 10 avec la même densité que 10 points en dimension 1, il faudrait 10^{10} points.

Conséquence

Les méthodes basées sur le **voisinage** (ex. k plus proches voisins, qui sera vu au cours 3 de régression) en souffrent particulièrement : à mesure que N augmente, les « voisins » s'éloignent.

- Un autre lien entre **volume de données** et **complexité** : plus le nombre de features est grand, plus il faut de données pour bien généraliser.

Mise en œuvre

1 Introduction

2 Approche générale

3 Mise en œuvre

4 Formulation & Optimisation

5 Généralisation et évaluation

Les données d'entrée

La majorité des algorithmes de Machine Learning utilisent une **représentation vectorielle** des données ($\mathbf{x} \in \mathbb{R}^N$)

Problème de représentation

Types de données "faciles" :

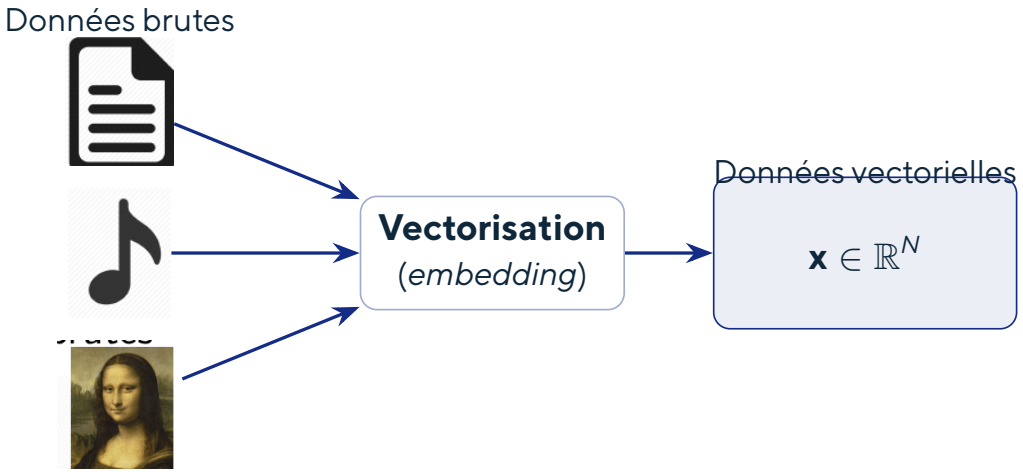
- Données quantitatives

Types de données "problématique" :

- Données catégorielles
- Textes
- Images

Extraction de caractéristiques

Le but est alors de convertir des données brutes en données vectorielles. On parle alors de **vectorisation** (ou *embedding* en anglais)



En général, les données brutes sont inexploitables

Texte, image, son, catégories...doivent tous être convertis en vecteurs numériques avant de pouvoir être utilisés par un algorithme d'apprentissage. Les diapositives suivantes détaillent comment, selon le type de donnée.

Qu'est-ce qu'une bonne représentation ?

Vectoriser n'est pas juste une formalité : le choix de la représentation a un impact **direct** sur ce que le modèle peut apprendre.

Taille fixe

Quelle que soit la taille de la donnée brute (un mot ou un roman, une petite ou grande image), le vecteur produit doit avoir une **dimension constante** N — la plupart des algorithmes l'exigent.

Préserver l'information utile

Ne pas jeter ce qui permet de distinguer les exemples entre eux vis-à-vis de la tâche visée.

La géométrie du vecteur doit avoir du sens

Deux données **similaires** doivent donner des vecteurs **proches** ; l'encodage ne doit pas introduire de fausse relation (ordre, distance, magnitude) qui n'existe pas dans la réalité.

Représenter des données catégorielles

Exemple : les tailles de T-shirts

Catégorie	Représentation vectorielle
XS	$x = (1, 0, 0, 0, 0)$
S	$x = (0, 1, 0, 0, 0)$
M	$x = (0, 0, 1, 0, 0)$
L	$x = (0, 0, 0, 1, 0)$
XL	$x = (0, 0, 0, 0, 1)$

Pourquoi pas simplement XS=1, S=2, ..., XL=5 ?

Coder ainsi impose une structure numérique qui n'a pas forcément de sens : le modèle interpréterait l'**écart** et la **magnitude** entre catégories (ex. « XL vaut 5 fois XS », « la distance S–M vaut exactement la distance M–L »). Pour une catégorie sans ordre naturel (couleur, type de document...), c'est **faux**.

La solution : le codage one-hot

Une **base canonique** : chaque catégorie est équidistante de toutes les autres, aucune relation d'ordre ou de magnitude n'est imposée — c'est au modèle d'apprendre une éventuelle relation à partir des données.

Représenter du texte

Un document texte (taille variable) doit devenir un vecteur de taille fixe.

Quelques caractéristiques possibles

- **Nombre de mots**, longueur moyenne des mots... : simples, mais perdent beaucoup d'information.
- **Fréquence des mots** (*bag-of-words*) : un vecteur indexé par un vocabulaire fixe, chaque case compte les occurrences — détaillé au cours 2.
- *etc.*

Lien avec « une bonne représentation »

La taille du vecteur (le vocabulaire) est fixée **à l'avance** et reste la même quelle que soit la longueur du texte d'origine.

Représenter une image et un son

Pour l'image

- **Histogramme des couleurs** : compte les pixels par plage de teinte — taille fixe, indépendante de la résolution de l'image.
- **Histogramme du gradient** : capture les contours/formes plutôt que les couleurs brutes.
- *etc.*

Pour le son

- **Spectre des fréquences** : décompose le signal en énergie par bande de fréquence, sur une fenêtre de temps fixe.
- *etc.*

Point commun

Dans les deux cas : on résume un signal de taille variable par un vecteur de taille fixe, en gardant l'information la plus **discriminante** pour la tâche visée.

Formulation & Optimisation

1 Introduction

2 Approche générale

3 Mise en œuvre

4 Formulation & Optimisation

5 Généralisation et évaluation

Notation mathématique

Formulation de prédicteur :

Fonction de prédiction :

$$\hat{y} = f_{\mu,\lambda}(\mathbf{x})$$

avec

- $\mathbf{x} \in \mathbb{R}^N$: le vecteur de données;
- $\hat{y} \in \mathbb{R}$ ou $\in \{-1, 1\}$: la valeur predite;
- μ : les paramétrés d'apprentissage;
- λ : les hyper-paramètres.

Cas générique

La forme exacte de $f_{\mu,\lambda}$ dépend de la méthode choisie (régression, régression logistique, discrimination bayésienne...). On garde ici une notation **générique** – un exemple complet sera traité en fin de cours.

Notation mathématique

Formulation de l'apprentissage :

Fonction d'apprentissage :

$$g(f_{\mu,\lambda}, \{\mathcal{X}, \mathcal{Y}\})$$

avec

- $f_{\mu,\lambda}$: la fonction de prédiction ;
- $\mathcal{X}$: ensemble des données d'apprentissages ;
- $\mathcal{Y}$: ensemble des prédictions attendues.

Cas générique

g mesure à quel point $f_{\mu,\lambda}$ se trompe sur $\{\mathcal{X}, \mathcal{Y}\}$ – sa forme exacte dépend elle aussi de la méthode (là encore, exemple complet en fin de cours). C'est la version calculable du **risque empirique** R_{emp} vu en Approche générale, appliquée à une fonction de prédiction paramétrée.

Optimisation

Définition

L'optimisation est une branche des mathématiques cherchant à trouver le minimum ou maximum d'une fonction

Application en apprentissage :

Apprendre consiste alors à rechercher le minimum de la fonction d'apprentissage g (vue à l'instant), en fonction des paramètres :

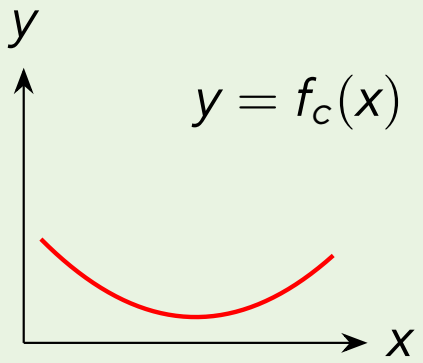
$$\mu^* = \arg\min_{\mu} g(f_{\mu,\lambda}, \{\mathcal{X}, \mathcal{Y}\})$$

μ^* sont les paramètres d'apprentissage optimum

Deux grandes catégories de problème d'optimisation

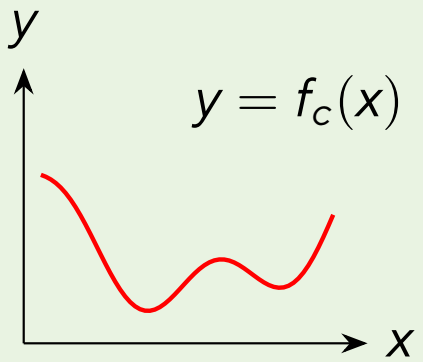
L'optimisation convexe

- **Simples** à analyser et à résoudre
- Algorithme **générique efficace** pour trouver la solution
- unique minimum global
  $\Rightarrow$ **unique** solution



L'optimisation non convexe

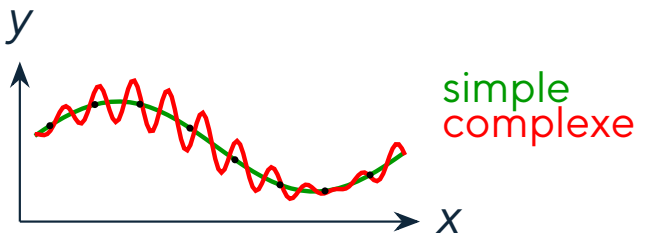
- **Difficile** à analyser et à résoudre
- **Pas** d'algorithme générique efficace
- Multiple minimum globaux et locaux
  $\Rightarrow$ **Plusieurs** solutions possibles !



Multiplicité des solutions et rasoir d'Ockham

Plusieurs solutions, une seule à choisir

- Non-convexité ⇒ plusieurs minima (vu à l'instant).
- Même en convexe : trop de paramètres / trop peu de données ⇒ une **infinité** de solutions à erreur d'apprentissage égale.



Deux fonctions passent par les mêmes points d'exemple

Rasoir d'Ockham

Entre plusieurs solutions qui expliquent aussi bien les données, préférer la plus **simple** (moins de variables, coefficients de faible amplitude).

Deux leviers, en pratique

Choisir directement un modèle **suffisamment simple** (mais pas trop — sous-apprentissage), **ou** garder un modèle riche mais **régulariser** : pénaliser la norme des paramètres μ (ex. $\|\mu\|_2^2$) dans le critère à minimiser. Les deux leviers seront comparés sur un exemple complet en fin de cours.

Comment trouve-t-on μ^* en pratique ?

Rappel

$$\mu^* = \arg\min_{\mu} g(f_{\mu,\lambda}, \{\mathcal{X}, \mathcal{Y}\})$$

En pratique, ce n'est pas trivial

- Pas de formule fermée en général
- La fonction à minimiser peut être non convexe
- Un modèle peut avoir un très grand nombre de paramètres

Il faut une méthode de recherche itérative

$\rightarrow$ c'est l'objet du **prochain TD** : la **descente de gradient**

Généralisation et évaluation

1 Introduction

2 Approche générale

3 Mise en œuvre

4 Formulation & Optimisation

5 Généralisation et évaluation

Taux d'erreur - Matrice de Confusion

Critère de performance pour la catégorielles

Le critère utilisé le plus courant est le **taux d'erreur** :

$$\text{taux d'erreur}(\hat{y}) = \frac{\text{nombre de fausse prediction}}{\text{nombre de prediction}}$$

Attention critère trop générique et trompeur !

Problèmes du taux d'erreur

- Base d'exemple déséquilibrer (99% d'exemple positif)
  - Que signifie un taux d'erreur de 1% ?
- Il y a 1000 catégories
  - Est-ce bien d'avoir un taux d'erreur de 50% ?
- Application un diagnostic maligne/bénigne pour une tumeur
  - Est-ce suffisant de savoir que l'on a un taux d'erreur de 5% ?

Taux d'erreur - Matrice de Confusion

Matrice de Confusion

La matrice des erreurs de prédiction de chaque catégorie de chaque catégorie

Pour deux catégories (Vrai/Faux)

		Prédit	
		Faux	Vrai
Vérité	Faux	TN	FP
	Vrai	FN	TP

- TP : Vrai Positif (*True Positive*)
- TN : Vrai Négatif (*True Negative*)
- FP : Faux Positif (*False Positive*)
- FN : Faux Négatif (*False Negative*)

Taux d'erreur - Matrice de Confusion

Exemple : Deux diagnostics de tumeur

		Prédit	
		maligne	bénigne
Vérité	maligne	98	2
	bénigne	50	850

		Prédit	
		maligne	bénigne
Vérité	maligne	55	45
	bénigne	5	895

Quel est le meilleur test ?

Taux d'erreur - Matrice de Confusion

MNIST - yann.lecun.com/exdb/mnist/

Base de données de chiffres écrits à la main, elle regroupe 60000 images d'apprentissage et 10000 images de test

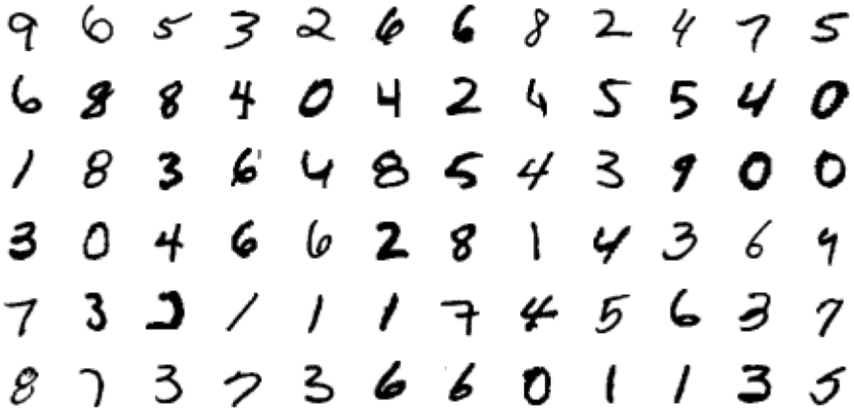


Figure : Exemple de la base MNIST

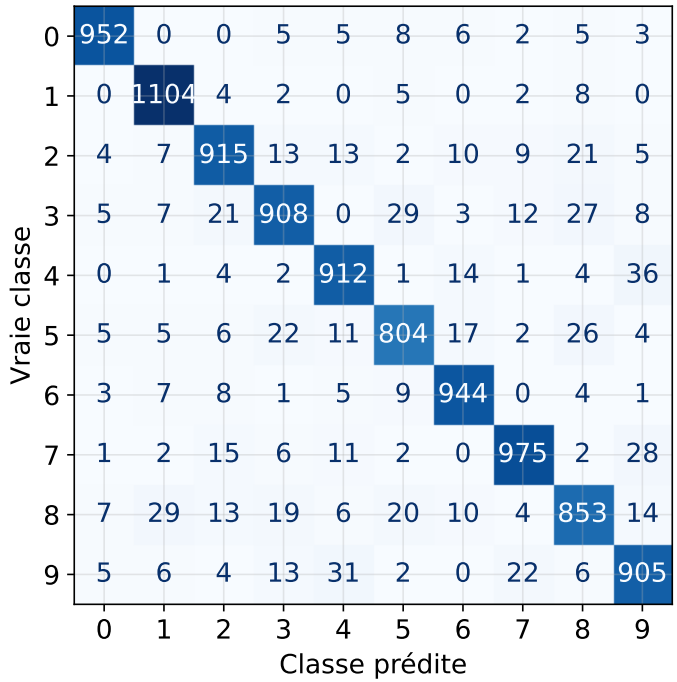


Figure : Matrice de confusion

Taux d'erreur - Matrice de Confusion

Pour deux catégories (Vrai/Faux)

		Prédit	
		Faux	Vrai
Vérité	Faux	TN	FP
	Vrai	FN	TP

Critères dérivés de la matrice de confusion

- **Exactitude** (*accuracy*) $= \dfrac{TP + TN}{TP + TN + FP + FN}$

  Proportion de prédictions correctes, toutes catégories confondues — trompeuse si les classes sont déséquilibrées (cf. exemple tumeur).

- **Précision** (*precision*) $= \dfrac{TP}{TP + FP}$

  Parmi les prédictions positives, combien le sont vraiment — utile quand un **faux positif** coûte cher (ex. filtre anti-spam).

- **Sensibilité** (*recall*, taux de vrais positifs) $= \dfrac{TP}{TP + FN}$

  Parmi les vrais positifs, combien sont détectés — utile quand **rater un positif** coûte cher (ex. diagnostic médical).

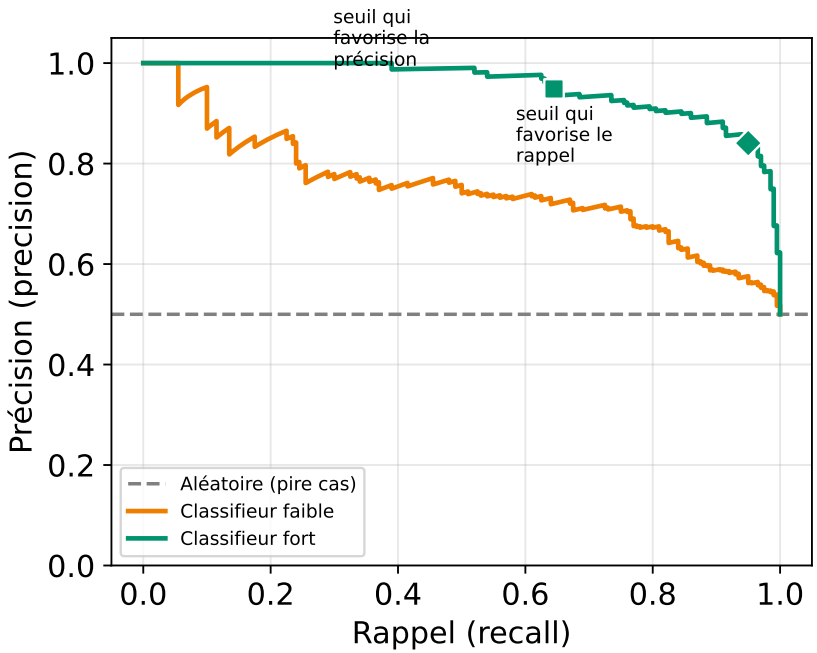
- **Spécificité** (taux de vrais négatifs) $= \dfrac{TN}{TN + FP}$

  Parmi les vrais négatifs, combien sont correctement écartés.

Limitation

Ne convient pas pour la classification multi-label (une donnée peut appartenir à plusieurs catégories).

Courbe rappel-précision



Principe

La plupart des classifieurs renvoient un **score**, pas juste Vrai/Faux : en faisant varier le seuil de décision, précision et rappel évoluent en sens opposé — la courbe trace ce compromis.

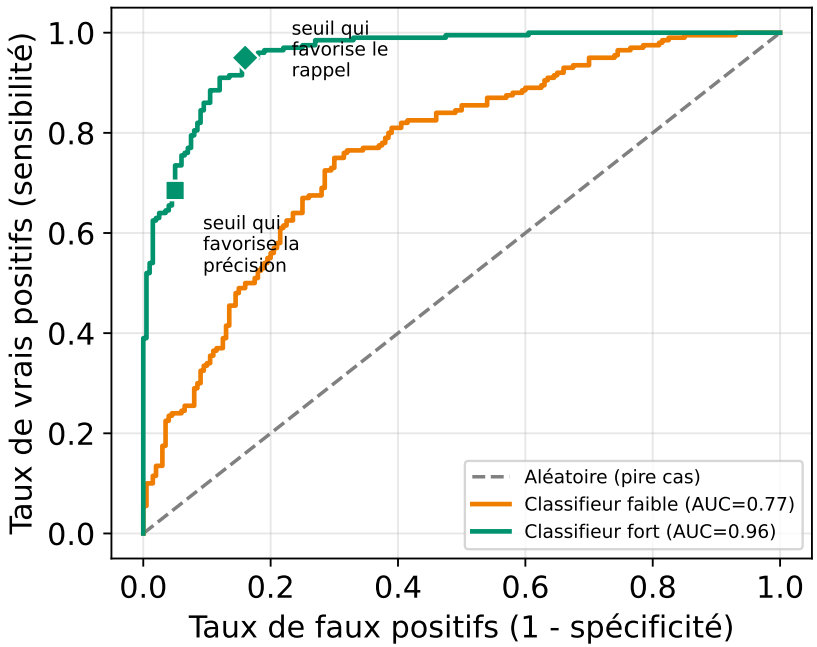
Avantages

Très informatif quand les classes sont **déséquilibrées**; permet de choisir un seuil adapté au coût réel des erreurs.

Limite

Ne se résume pas à un seul chiffre — on la résume parfois par son aire (**précision moyenne**).

Courbe ROC



Principe

Receiver Operating Characteristic : trace le taux de vrais positifs (sensibilité) contre le taux de faux positifs, pour tous les seuils de décision possibles.

Avantages

Compare des modèles indépendamment du seuil choisi; l'aire sous la courbe (**AUC**) résume la qualité globale en un chiffre (1 = parfait, 0.5 = aléatoire).

Limite

Peut donner une impression trop optimiste sur des classes très déséquilibrées — la courbe rappel-précision est alors plus parlante.

Score F_1 (et variantes)

Principe

$$F_1 = 2 \times \frac{\text{précision} \times \text{sensibilité}}{\text{précision} + \text{sensibilité}}$$

La moyenne harmonique de la précision et de la sensibilité : résume les deux en un seul chiffre.

Le score F_β

Généralise F_1 en pondérant : $\beta > 1$ privilégie la sensibilité, $\beta < 1$ privilégie la précision — utile quand l'un des deux compte plus que l'autre.

Pour le multi-label

Se décline par label (précision/rappel par label), avant d'être moyenné sur l'ensemble des catégories.

Autres critères

- **Coefficient de corrélation de Matthews** : résume toute la matrice de confusion en un seul chiffre entre -1 et 1 — robuste même sur des classes très déséquilibrées.
- **Précision moyenne** (*average precision*) : aire sous la courbe rappel-précision — fonctionne en binaire comme en multi-label (mais pas en multi-classes).

No Free Lunch

Théorème du No Free Lunch (Wolpert & Macready)

En moyenne sur **tous les problèmes possibles**, aucun algorithme de machine learning n'est meilleur qu'un autre — pas même meilleur qu'un algorithme aléatoire.

En pratique

Les problèmes réels ne sont **pas** « tous les problèmes possibles » : ils ont une structure. C'est pour **ce type** de problèmes que certains algorithmes marchent mieux que d'autres.

D'où l'intérêt de connaître plusieurs méthodes

Il n'existe pas de méthode universellement meilleure : savoir laquelle convient à quel contexte est essentiel $\rightarrow$ objet des **prochains cours** (régression, régression logistique, discrimination bayésienne, ...).

Exemple final — la régression polynomiale, formellement

On reprend l'exemple utilisé depuis le début du cours, formulé cette fois **complètement** :

Fonction de prédiction

$$\hat{y} = f_{\mu,\lambda}(x) = \sum_{k=0}^{D} a_k x^k \qquad \mu = \{a_0, \ldots, a_D\} \quad \lambda = \{D, \alpha\}$$

Fonction objectif (minimisée sur train)

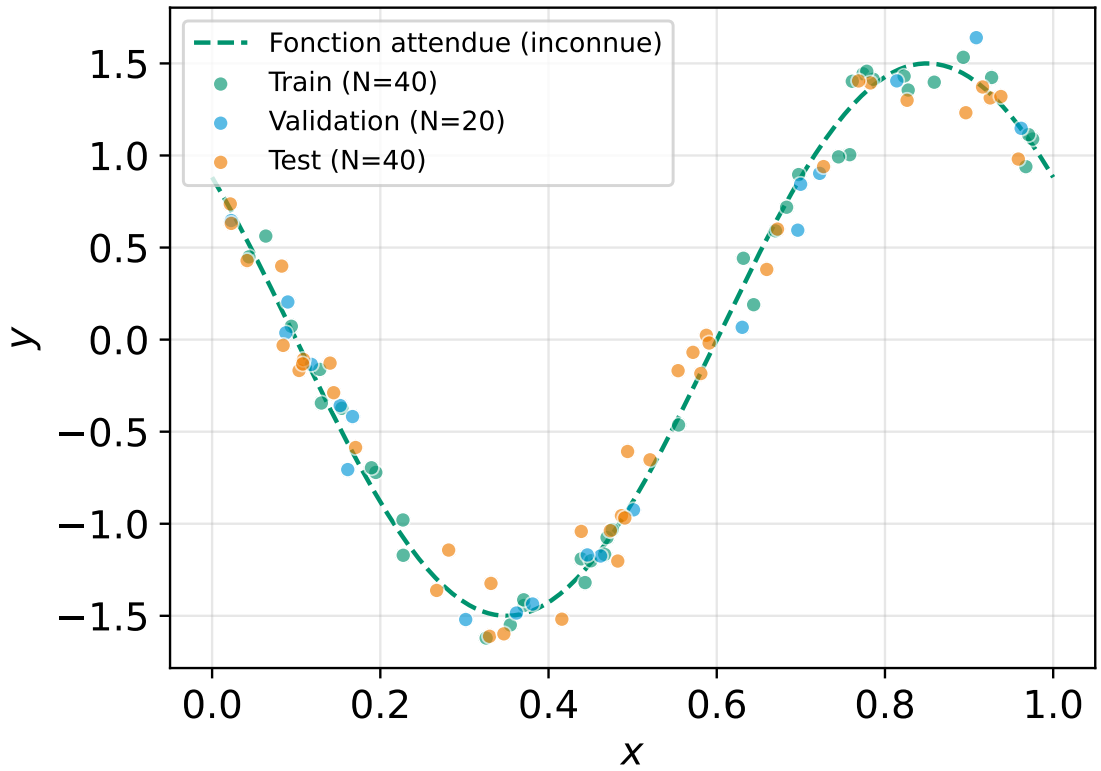
$$\mu^* = \arg\min_{\mu} \underbrace{\sum_{i=1}^{P} (f_{\mu,\lambda}(x_i) - y_i)^2}_{\text{erreur}} + \underbrace{\alpha \|\mu\|_2^2}_{\text{régularisation}}$$

Paramètres vs hyperparamètres, ici

- **Paramètres** μ : les coefficients $a_0, \ldots, a_D$ — appris automatiquement sur train.
- **Hyperparamètres** λ : le degré D **et** la force de régularisation α — choisis sur validation.

Exemple final — les données

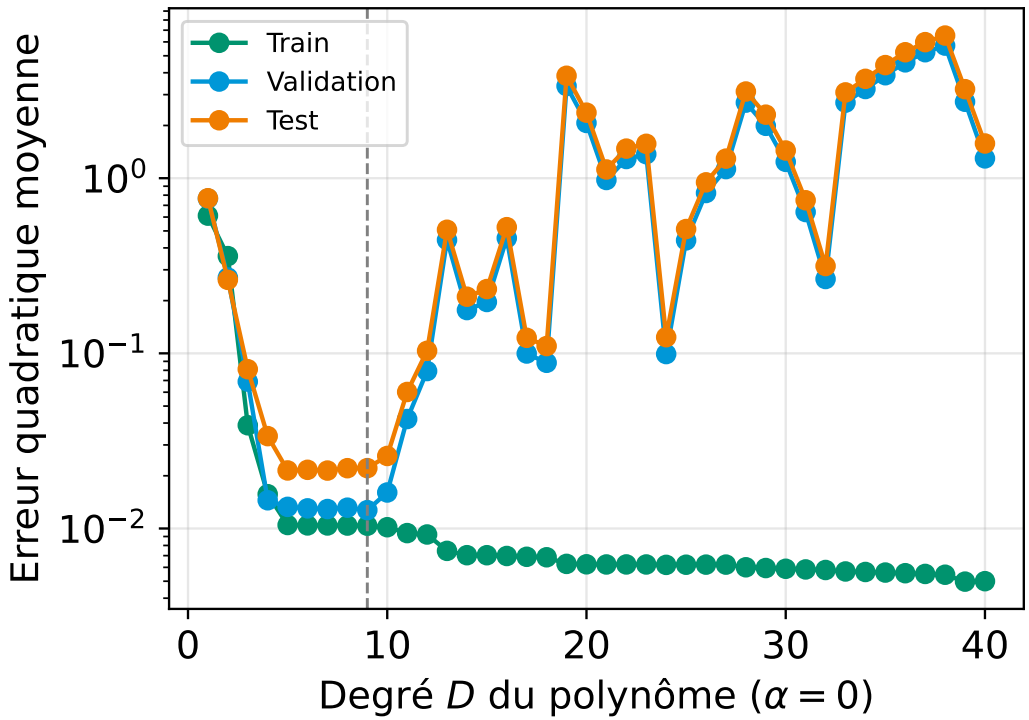
On tire des points, bruités autour d'une fonction inconnue, et on découpe en train / validation / test.



- **Train** : apprend μ pour chaque (D, α) essayé.
- **Validation** : choisit (D, α).
- **Test** : jamais touché avant la toute fin.

Exemple final — choisir le degré ($\alpha = 0$)

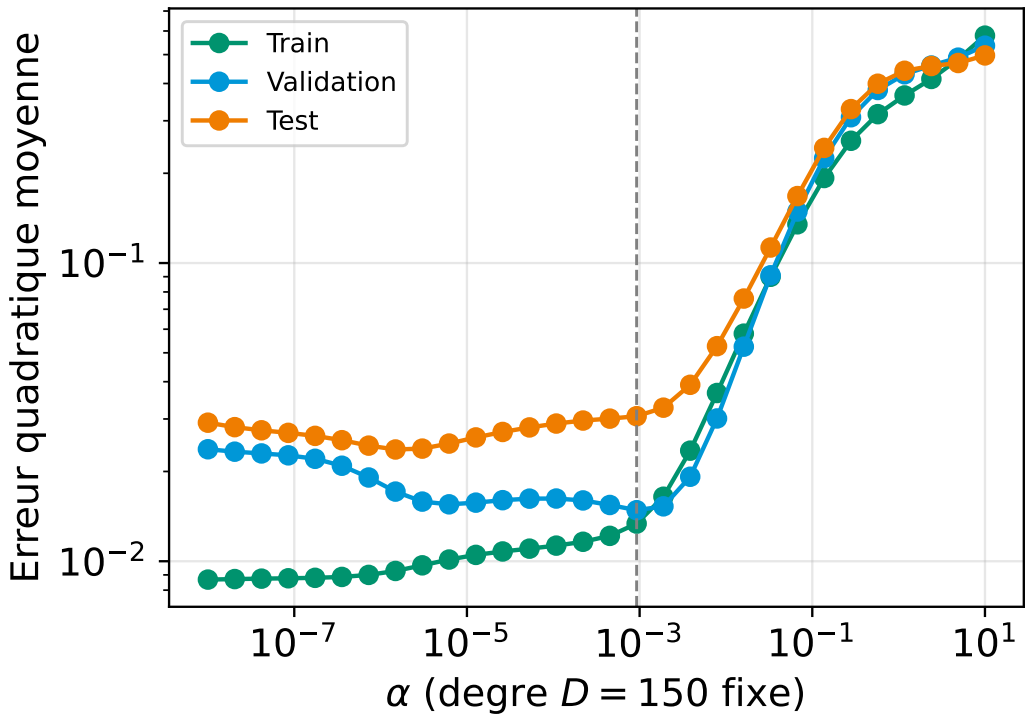
Premier levier : sans régularisation ($\alpha = 0$), on fait varier le degré D.



- L'erreur train baisse toujours avec D : elle ne permet pas de choisir.
- L'erreur validation forme un creux $\rightarrow$ c'est **elle** qui fixe D^*.
- L'erreur test (jamais utilisée pour choisir) confirme *a posteriori* que D^* généralise bien.

Exemple final — choisir la régularisation

Second levier : **sans** changer D (fixé volontairement très élevé, $D = 150$), on fait varier α.



- α trop faible $\rightarrow$ sur-apprentissage (comme un D trop grand).
- α trop fort $\rightarrow$ sous-apprentissage (comme un D trop petit).
- Encore une fois, c'est l'erreur validation qui fixe α^* — pas train, pas test.

Exemple final — bilan

Deux leviers, un seul objectif

Contrôler la **complexité effective** du modèle : soit directement (D), soit indirectement en contraignant l'amplitude des paramètres (α). Dans les deux cas, le choix se fait **sur validation**, jamais sur test.

Ce que cet exemple a mis bout à bout

Formulation $(\mu, \lambda) \rightarrow$ optimisation $(\mu^*) \rightarrow$ découpage train/validation/test $\rightarrow$ sous/sur-apprentissage $\rightarrow$ rasoir d'Ockham et régularisation.

Annexes

6 Annexes

Erreur quadratique moyenne

L'erreur quadratique moyenne est un critère de performance très souvent utilisé quand l'on cherche à estimé une **quantitative** :

$$\text{EQM}(\hat{y}) = \mathbb{E}\left[(\hat{y} - y)^2\right]$$

Elle se décompose en deux termes :

$$\mathbb{E}\left[(\hat{y} - y)^2\right] = \underbrace{\mathbb{E}\left[(\mathbb{E}(\hat{y}) - y)^2\right]}_{\text{Biais}(\hat{y})^2} + \underbrace{\mathbb{E}\left[(\mathbb{E}(\hat{y}) - \hat{y})^2\right]}_{\text{Var}(\hat{y})}$$

Ces deux critères sont également intéressante à étudier :

- Une **biais important** signifie que le modèle sous-jacent est trop simple (**Sous-apprentissage**)
- Une **variance importante** signifie que le modèle sous-jacent est trop complexes

Erreur quadratique moyenne

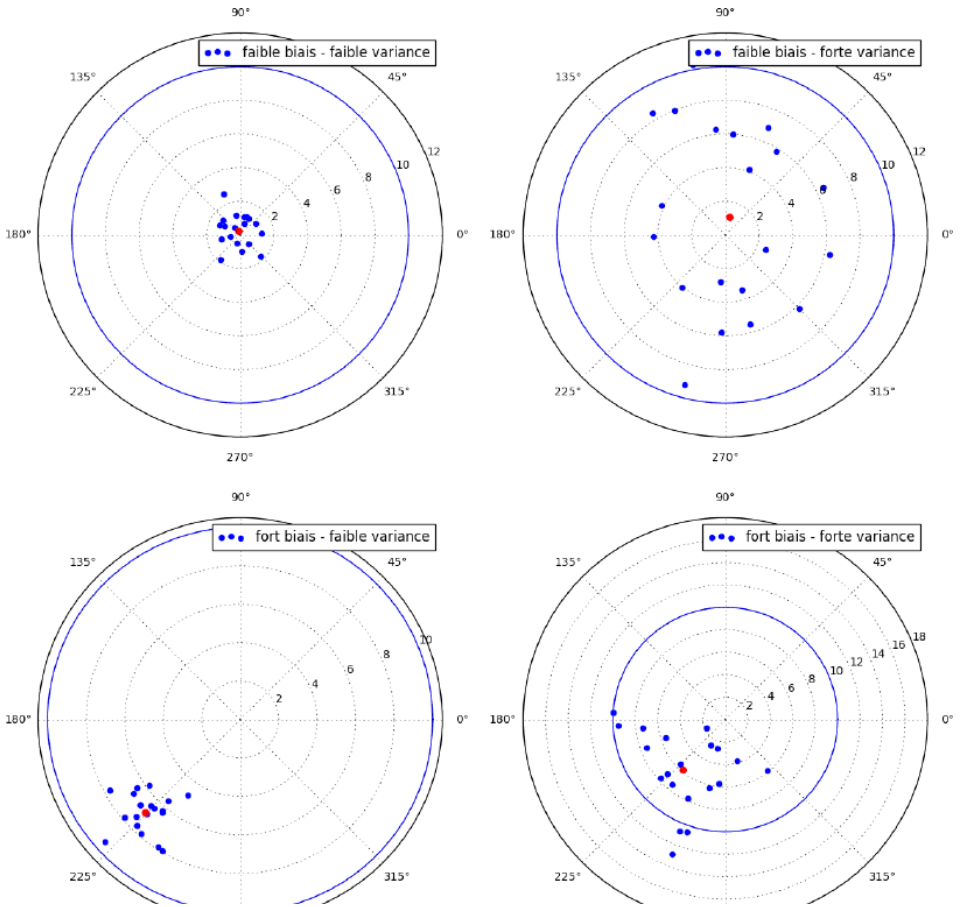


Figure : signification des termes de biais et variance

- Centre de la cible = valeur vraie ; chaque point = une prédiction obtenue avec un jeu d'entraînement différent.
- **Biais** = décalage du barycentre par rapport au centre.
- **Variance** = dispersion des points entre eux.
- Objectif : faible biais **et** faible variance (en haut à gauche).